CO2 - ASSESSMENT TOOL 1

NAME: SIDDHI SRIVASTAVA. P

REG.NO:192521454

COURSE CODE: CSA0503

COURSE NAME: DATABASE MANAGEMENT SYSTEMS

1. Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.

Sol:

Employee Table

EmpID	EmpName	Salary	DeptID
101	John	50000	D1
102	Alice	65000	D2
103	David	45000	D1
104	Emma	70000	D3

Department Table

DeptID	DeptName
D1	HR
D2	IT
D3	Finance

1. Selection (σ)
Retrieves rows satisfying a condition.
Query
Employees having salary greater than 50000.
Relational Algebra
σ Salary > 50000 (Employee)

EmpID	EmpName	Salary	DeptID
102	Alice	65000	D2
104	Emma	70000	D3

2. Projection (π) : Retrieves selected columns only.

Query : Display employee names.

Relational Algebra

π EmpName(Employee)

EmpName
John
Alice
David
Emma

3. Union (∪)

Suppose another relation exists.

EmpID	EmpName
105	Chris
106	Sophia

4. Relational Algebra

π EmpName(Employee)
∪
π EmpName(Employee_New)

5. Cartesian Product (×)

Combine every employee with every department.

Relational Algebra

Employee × Department

If Employee has 4 records and Department has 3 records,
Total tuples = 4 × 3 = 12

6. Join (⋈)

Display employee names with department names.

Relational Algebra

Employee ⋈ Employee.DeptID = Department.DeptID Department

Employee Department
John HR

Employee Department
Alice IT
David HR
Emma Finance

2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

Sol:

```
mysql> CREATE TABLE Department(
    -> DeptID INT PRIMARY KEY,
    -> DeptName VARCHAR(50) NOT NULL UNIQUE
    -> );
Query OK, 0 rows affected (0.21 sec)

mysql>
mysql> CREATE TABLE Student(
    -> SID INT PRIMARY KEY,
    -> Name VARCHAR(50) NOT NULL,
    -> DOB DATE NOT NULL,
    -> DeptID INT,
    -> FOREIGN KEY (DeptID)
    -> REFERENCES Department(DeptID)
    -> ON DELETE CASCADE
    -> ON UPDATE CASCADE
    -> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> INSERT INTO Student
    -> VALUES
    -> (101,'Rahul','2004-01-12',1),
    -> (102,'Sneha','2004-08-25',2),
    -> (103,'Karan','2003-11-10',3);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> desc Student;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| SID    | int         | NO   | PRI | NULL    |       |
| Name   | varchar(50) | NO   |     | NULL    |       |
| DOB    | date        | NO   |     | NULL    |       |
| DeptID | int         | YES  | MUL | NULL    |       |
+--------+-------------+------+-----+---------+-------+
```

```
mysql> desc department;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| DeptID   | int         | NO   | PRI | NULL    |       |
| DeptName | varchar(50) | NO   | UNI | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

DDL (Data Definition Language) is used to create and modify database objects. The CREATE TABLE statement defines the table structure, while constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE ensure data integrity and maintain relationships between tables.

3. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a Sales dataset.

Sol:

SaleID	Product	Department	Amount
1	Laptop	Electronics	60000
2	Mouse	Electronics	800
3	Chair	Furniture	5000
4	Table	Furniture	8000
5	Monitor	Electronics	12000

```
mysql> CREATE TABLE Sales (
    ->     SaleID INT PRIMARY KEY,
    ->     Product VARCHAR(50) NOT NULL,
    ->     Department VARCHAR(50) NOT NULL,
    ->     Amount DECIMAL(10,2) NOT NULL
    -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> INSERT INTO Sales (SaleID, Product, Department, Amount)
    -> VALUES
    -> (1, 'Laptop', 'Electronics', 60000),
    -> (2, 'Mouse', 'Electronics', 800),
    -> (3, 'Chair', 'Furniture', 5000),
    -> (4, 'Table', 'Furniture', 8000),
    -> (5, 'Monitor', 'Electronics', 12000),
    -> (6, 'Keyboard', 'Electronics', 1500),
    -> (7, 'Sofa', 'Furniture', 25000),
    -> (8, 'Printer', 'Electronics', 18000);
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT COUNT(*) AS TotalSales
    -> FROM Sales;
+------------+
| TotalSales |
+------------+
|          8 |
+------------+
1 row in set (0.03 sec)
```

```
mysql> SELECT SUM(Amount) AS TotalSalesAmount
    -> FROM Sales;
+-------------------+
| TotalSalesAmount |
+-------------------+
|         130300.00 |
+-------------------+
1 row in set (0.01 sec)

mysql> SELECT AVG(Amount) AS AverageSales
    -> FROM Sales;
+---------------+
| AverageSales |
+---------------+
| 16287.500000 |
+---------------+
1 row in set (0.01 sec)
```

```
mysql> SELECT MAX(Amount) AS HighestSale
    -> FROM Sales;
+--------------+
| HighestSale |
+--------------+
|     60000.00 |
+--------------+
1 row in set (0.01 sec)

mysql> SELECT MIN(Amount) AS LowestSale
    -> FROM Sales;
+-------------+
| LowestSale |
+-------------+
|      800.00 |
+-------------+
1 row in set (0.00 sec)
```

```
mysql> SELECT Department,
    ->        COUNT(*) AS NumberOfSales,
    ->        SUM(Amount) AS TotalAmount
    -> FROM Sales
    -> GROUP BY Department;
+-------------+---------------+-------------+
| Department  | NumberOfSales | TotalAmount |
+-------------+---------------+-------------+
| Electronics |             5 |    92300.00 |
| Furniture   |             3 |    38000.00 |
+-------------+---------------+-------------+
2 rows in set (0.01 sec)
```

```
mysql> SELECT Department,
    ->        SUM(Amount) AS TotalAmount
    -> FROM Sales
    -> GROUP BY Department
    -> HAVING SUM(Amount) > 50000;
+-------------+-------------+
| Department  | TotalAmount |
+-------------+-------------+
| Electronics |    92300.00 |
+-------------+-------------+
1 row in set (0.01 sec)
```

4. Write a correlated sub-query to find employees earning more than the average salary of their own department.

Sol:

```
mysql> CREATE TABLE Employee (
    ->     EmpID INT PRIMARY KEY,
    ->     EmpName VARCHAR(50) NOT NULL,
    ->     Salary DECIMAL(10,2),
    ->     DeptID INT
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Employee (EmpID, EmpName, Salary, DeptID)
    -> VALUES
    -> (101,'John',50000,1),
    -> (102,'Alice',65000,2),
    -> (103,'David',45000,1),
    -> (104,'Emma',70000,3),
    -> (105,'Sophia',60000,2),
    -> (106,'Michael',40000,3);
Query OK, 6 rows affected (0.02 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

Corelated Subquery:

```
mysql> SELECT EmpID, EmpName, Salary, DeptID
    -> FROM Employee e
    -> WHERE Salary >
    -> (
    ->     SELECT AVG(Salary)
    ->     FROM Employee
    ->     WHERE DeptID = e.DeptID
    -> );
+-------+---------+----------+--------+
| EmpID | EmpName | Salary   | DeptID |
+-------+---------+----------+--------+
|   101 | John    | 50000.00 |      1 |
|   102 | Alice   | 65000.00 |      2 |
|   104 | Emma    | 70000.00 |      3 |
+-------+---------+----------+--------+
3 rows in set (0.01 sec)
```

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

Sol: Employee table

Emp_ID	Emp_Name	Dept	Salary
101	Arun	HR	45000
102	Priya	IT	60000
103	Rahul	Finance	55000
104	Neha	IT	50000
105	Kiran	Sales	48000

Project table:

Project_ID	Project_Name	Emp_ID
P101	Payroll System	101
P102	Website	102
P103	Banking App	103
P104	Mobile App	106

INNER JOIN :

```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM Employee E
    -> INNER JOIN Project P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+----------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name   |
+--------+----------+------------+----------------+
|    101 | Arun     | P101       | Payroll System |
|    102 | Priya    | P102       | Website        |
|    103 | Rahul    | P103       | Banking App    |
+--------+----------+------------+----------------+
3 rows in set (0.04 sec)
```

LEFT OUTER JOIN:

```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM Employee E
    -> LEFT OUTER JOIN Project P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+----------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name   |
+--------+----------+------------+----------------+
|    101 | Arun     | P101       | Payroll System |
|    102 | Priya    | P102       | Website        |
|    103 | Rahul    | P103       | Banking App    |
|    104 | Neha     | NULL       | NULL           |
|    105 | Kiran    | NULL       | NULL           |
+--------+----------+------------+----------------+
5 rows in set (0.01 sec)
```

RIGHT OUTER JOIN:

```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM Employee E
    -> RIGHT OUTER JOIN Project P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+----------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name   |
+--------+----------+------------+----------------+
|    101 | Arun     | P101       | Payroll System |
|    102 | Priya    | P102       | Website        |
|    103 | Rahul    | P103       | Banking App    |
|   NULL | NULL     | P104       | Mobile App     |
+--------+----------+------------+----------------+
4 rows in set (0.00 sec)
```

FULL OUTER JOIN:

```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM Employee E
    -> LEFT JOIN Project P
    -> ON E.Emp_ID = P.Emp_ID
    ->
    -> UNION
    ->
    -> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM Employee E
    -> RIGHT JOIN Project P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+----------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name   |
+--------+----------+------------+----------------+
|    101 | Arun     | P101       | Payroll System |
|    102 | Priya    | P102       | Website        |
|    103 | Rahul    | P103       | Banking App    |
|    104 | Neha     | NULL       | NULL           |
|    105 | Kiran    | NULL       | NULL           |
|   NULL | NULL     | P104       | Mobile App     |
+--------+----------+------------+----------------+
6 rows in set (0.06 sec)
```

CROSS JOIN:

```
mysql> SELECT E.Emp_Name, P.Project_Name
    -> FROM Employee E
    -> CROSS JOIN Project P;
+----------+----------------+
| Emp_Name | Project_Name   |
+----------+----------------+
| Arun     | Mobile App     |
| Arun     | Banking App    |
| Arun     | Website        |
| Arun     | Payroll System |
| Priya    | Mobile App     |
| Priya    | Banking App    |
| Priya    | Website        |
| Priya    | Payroll System |
| Rahul    | Mobile App     |
| Rahul    | Banking App    |
| Rahul    | Website        |
| Rahul    | Payroll System |
| Neha     | Mobile App     |
| Neha     | Banking App    |
| Neha     | Website        |
| Neha     | Payroll System |
| Kiran    | Mobile App     |
| Kiran    | Banking App    |
| Kiran    | Website        |
| Kiran    | Payroll System |
+----------+----------------+
20 rows in set (0.04 sec)
```

6. Create a view 'DeptSalaryView' that displays department name, total employees, and average salary.

Sol:

```
mysql> CREATE VIEW DeptSalaryView AS
    -> SELECT
    ->     Department,
    ->     COUNT(*) AS Total_Employees,
    ->     AVG(Salary) AS Average_Salary
    -> FROM Employee
    -> GROUP BY Department;
Query OK, 0 rows affected (0.11 sec)

mysql> SELECT * FROM DeptSalaryView;
+------------+-----------------+----------------+
| Department | Total_Employees | Average_Salary |
+------------+-----------------+----------------+
| HR         |               1 |   45000.000000 |
| IT         |               2 |   55000.000000 |
| Finance    |               1 |   55000.000000 |
| Sales      |               1 |   48000.000000 |
+------------+-----------------+----------------+
4 rows in set (0.02 sec)
```

```
mysql> SELECT *
    -> FROM DeptSalaryView
    -> WHERE Average_Salary > 50000;
+------------+-----------------+----------------+
| Department | Total_Employees | Average_Salary |
+------------+-----------------+----------------+
| IT         |               2 |   55000.000000 |
| Finance    |               1 |   55000.000000 |
+------------+-----------------+----------------+
2 rows in set (0.01 sec)
```

7. Construct a Relational Algebra expression

Question: Find names of students who are enrolled in all courses offered.

Assume the relations:

- Student(Student_ID, Student_Name)
- Course(Course_ID, Course_Name)
- Enrollment(Student_ID, Course_ID)

Relational Algebra Expression:

πStudent_Name(Student $\bowtie$(

πStudent_ID(Enrollment $\div$ πCourse_ID(Course))))

8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement

Sol:

```
mysql> INSERT INTO Employee
    -> VALUES
    -> (106,'Riya','Marketing',52000);
Query OK, 1 row affected (0.05 sec)

mysql> UPDATE Employee
    -> SET Salary = Salary + 5000
    -> WHERE Emp_ID = 106;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> INSERT INTO Employee
    -> VALUES
    -> (101,'Anil','HR',45000);
ERROR 1062 (23000): Duplicate entry '101' for key 'employee.PRIMARY'
```

9. Find all employees who work in the IT department and earn more than 50000.

Sol:

Tuple relational calculus:

{ t | Employee(t) ∧ t.Department = 'IT' ∧ t.Salary > 50000 }

Equivalent SQL

SELECT *FROM EmployeeWHERE Department='IT'AND Salary>50000;

10. Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, subqueries, and views Schema.

Sol:

Creating tables:

```
mysql> select * from Passenger;
+--------------+----------------+------------+
| Passenger_ID | Passenger_Name | Phone      |
+--------------+----------------+------------+
|            1 | Arun           | 9876543210 |
|            2 | Priya          | 9123456780 |
|            3 | Rahul          | 9988776655 |
+--------------+----------------+------------+
3 rows in set (0.00 sec)

mysql> select *from Flight;
+-----------+-------------+---------+-------------+
| Flight_ID | Flight_Name | Source  | Destination |
+-----------+-------------+---------+-------------+
|       101 | IndiGo      | Chennai | Delhi       |
|       102 | Air India   | Mumbai  | Bangalore   |
+-----------+-------------+---------+-------------+
2 rows in set (0.04 sec)

mysql> select*from Reservation;
+----------------+--------------+-----------+---------+
| Reservation_ID | Passenger_ID | Flight_ID | Seat_No |
+----------------+--------------+-----------+---------+
|           1001 |            1 |       101 | A1      |
|           1002 |            2 |       102 | B2      |
|           1003 |            3 |       101 | A3      |
+----------------+--------------+-----------+---------+
3 rows in set (0.00 sec)
```

INNER JOIN:

```
mysql> SELECT p.Passenger_Name,
    ->        f.Flight_Name,
    ->        r.Seat_No
    -> FROM Passenger p
    -> JOIN Reservation r
    -> ON p.Passenger_ID=r.Passenger_ID
    -> JOIN Flight f
    -> ON r.Flight_ID=f.Flight_ID;
+----------------+-------------+---------+
| Passenger_Name | Flight_Name | Seat_No |
+----------------+-------------+---------+
| Arun           | IndiGo      | A1      |
| Rahul          | IndiGo      | A3      |
| Priya          | Air India   | B2      |
+----------------+-------------+---------+
3 rows in set (0.01 sec)
```

LEFT JOIN:

```
mysql> SELECT p.Passenger_Name,
    ->        r.Reservation_ID
    -> FROM Passenger p
    -> LEFT JOIN Reservation r
    -> ON p.Passenger_ID=r.Passenger_ID;
+----------------+----------------+
| Passenger_Name | Reservation_ID |
+----------------+----------------+
| Arun           |           1001 |
| Priya          |           1002 |
| Rahul          |           1003 |
+----------------+----------------+
3 rows in set (0.00 sec)
```

SUBQUERY:

```
mysql> SELECT Passenger_Name
    -> FROM Passenger
    -> WHERE Passenger_ID IN
    -> (
    -> SELECT Passenger_ID
    -> FROM Reservation
    -> WHERE Flight_ID=
    -> (
    -> SELECT Flight_ID
    -> FROM Flight
    -> WHERE Destination='Delhi'
    -> )
    -> );
+-----------------+
| Passenger_Name  |
+-----------------+
| Arun            |
| Rahul           |
+-----------------+
2 rows in set (0.05 sec)
```

CREATE AND DISPLAY VIEW:

```
mysql> CREATE VIEW FlightDetails AS
    -> SELECT p.Passenger_Name,
    ->        f.Flight_Name,
    ->        f.Source,
    ->        f.Destination
    -> FROM Passenger p
    -> JOIN Reservation r
    -> ON p.Passenger_ID=r.Passenger_ID
    -> JOIN Flight f
    -> ON r.Flight_ID=f.Flight_ID;
Query OK, 0 rows affected (0.06 sec)

mysql> SELECT * FROM FlightDetails;
+----------------+-------------+---------+-------------+
| Passenger_Name | Flight_Name | Source  | Destination |
+----------------+-------------+---------+-------------+
| Arun           | IndiGo      | Chennai | Delhi       |
| Rahul          | IndiGo      | Chennai | Delhi       |
| Priya          | Air India   | Mumbai  | Bangalore   |
+----------------+-------------+---------+-------------+
3 rows in set (0.04 sec)
```